Clerkey — Project Requirements Document (PRD / SRS)

Version: 1.1 **Status:** Draft for hackathon build **Owner:** Mercy Munzenzi

---

1. Project Overview

Clerkey is a multi-tenant AI agent platform that helps small businesses respond to customer inquiries across messaging channels (WhatsApp, email, Instagram/Facebook) in real time. It checks the business's current, live state — stock on hand, service availability, capacity, current rates, or whatever fact matters for that industry — before confirming anything to a customer, escalates ambiguous or high-stakes conversations to a human, and improves its responses over time by learning from corrections a human makes to its drafts.

Clerkey is industry-agnostic by design. It is not a stock-management tool with a chat interface bolted on — it's a customer-response agent built around one general pattern that shows up in every business, whether or not that business sells physical goods: **some fact about the business goes stale over time, and confirming it accurately before answering a customer is what makes the difference between a good automated reply and a wrong one.**

Industry	What "business state" means for them
Retail / grocery / feed supply	Stock count of an item
Law firm	Whether new clients are being accepted; consultation rates
Software / dev agency	Open capacity for new projects; current service pricing
Consulting	Available hours this month; day rate

Each business (tenant) gets its own isolated workspace: their own business-state data, customer conversations, channel connections, and dashboard — all running on one shared Clerkey deployment rather than a separate install per business.

Built on Qwen Cloud (reasoning/generation) and deployed on Alibaba Cloud (compute, database, and infrastructure).

---

2. Business Objectives

- Reduce response time to customer inquiries from hours to seconds for small

businesses that can't staff 24/7 support, across any industry — not just product-based ones.
- Increase inquiry-to-conversion rate by giving customers accurate, immediate answers instead of losing them to slow replies.
- Remove the #1 adoption blocker for SMB AI tools: the burden of manually uploading and maintaining structured business data, whatever form that data takes for a given industry.
- Prove a productizable, multi-tenant SaaS architecture — not just a single-business demo — to demonstrate real scalability and open-source community potential across verticals.

---

3. User Roles

Role	Description	Access
Business owner / admin	Signs up, onboards their business, manages business-state data, connects channels, reviews escalations, views analytics	Full access to their own tenant workspace only
Staff member (future)	Can view/respond to escalations, cannot change business settings or billing	Scoped access within their tenant
End customer	Messages the business via WhatsApp/email/Instagram; never logs into anything	No dashboard access — interacts only via messaging channels
Platform admin (you, during hackathon)	Manages the overall Clerkey deployment, tenant provisioning, system health	Cross-tenant access for operations only, never for reading tenant business data

---

4. Scope

In scope (hackathon build)

- Multi-tenant backend with strict per-tenant data isolation
- Web dashboard for business owners (auth, onboarding, business-state view, conversation/ escalation review, basic analytics)

- One-time guided business info upload during onboarding (business profile, initial business-state data in whatever shape fits the tenant's industry, tone/policy preferences)
- Ongoing low-friction business-state updates via proactive check-in messages (WhatsApp) — the primary maintenance mechanism *after* onboarding, not a repeat upload flow
- Two messaging channels wired end-to-end for the demo: WhatsApp + Email (Instagram/ Facebook documented as an additional channel adapter, not required to fully ship in hackathon timeframe)
- Core agent loop: receive inquiry → retrieve context/business state → draft response → confidence check → auto-reply or escalate
- Memory system: per-tenant conversation history, preferences, and business-state facts, with human-correction feedback improving future responses
- Human-in-the-loop escalation via WhatsApp message to the owner (no separate approval UI required for MVP — dashboard can show escalation log/history)
- At least two seeded demo tenants from genuinely different industries (e.g., one product-based, one service-based) to demonstrate the platform is not stock-specific

Out of scope (explicitly, for this build)

- Billing/subscription system
- Staff role permissions beyond "owner" (structure allows for it later, not built now)
- Full CRM feature set (this is a response/business-state assistant, not a replacement CRM)
- LinkedIn channel integration (API access constraints — noted as future work)
- Voice/phone call channel
- Native mobile app (dashboard is web-only)

---

5. Functional Requirements

5.1 Tenant & Account Management

- FR-1: A new business can sign up, select or describe their industry/business type, and create an isolated tenant workspace.
- FR-2: Each tenant's data (conversations, business state, customers, settings) is fully isolated from every other tenant at the database and application layer.
- FR-3: Business owner can log into a web dashboard scoped to their tenant only.

5.2 Onboarding (One-Time Upload)

- FR-4: On first login, the owner completes a guided onboarding flow: business profile (name, description, policies, tone preferences), initial business-state data (see 5.3 for the general data shape — this may be a product catalog with quantities, a list of services with availability/rates, or a mix, depending on industry), and channel connection (WhatsApp Business number, email inbox).
- FR-5: This upload is designed as a one-time structured setup step — not the ongoing maintenance mechanism (see 5.4).
- FR-5a: The onboarding flow adapts its prompts based on the business type selected (e.g., "add your products and stock counts" for a retailer vs. "add your services and current availability" for a law firm or agency), rather than assuming every business sells countable items.

5.3 Business State Model (General)

- FR-5b: The system represents anything that can go stale about a business as a **business state item**: a named fact (e.g., "50kg maize bags," "new client intake," "mobile app development package"), a current value (a quantity, an available/unavailable flag, a price, or a short text status), and a last-confirmed timestamp. This is the one general data shape that covers stock, capacity, availability, and pricing use cases without needing separate systems per industry.

5.4 Ongoing Business-State Maintenance (Low-Friction Check-Ins)

- FR-6: After onboarding, the system does NOT require the owner to return to a dashboard or re-upload data to keep business-state data current.
- FR-7: The system proactively sends the owner short WhatsApp check-in messages when it detects a signal that a business-state item may be stale (e.g., repeated customer inquiries about that item, time elapsed since last confirmation, a quantity-based threshold approaching zero where applicable).
- FR-8: The owner can update the item with a one-line reply (e.g., "12 left," "yes still taking clients," "still $150/hr") without opening the dashboard; the system parses the reply and updates that business-state item.
- FR-9: The dashboard also allows direct manual edits to any business-state item for owners who prefer it, but this is optional, not required.

5.5 Customer Inquiry Handling

- FR-10: The system ingests inquiries from WhatsApp and Email (Instagram/Facebook as a stretch channel), normalizing each into a common internal message format.
- FR-11: For each inquiry, the agent retrieves relevant tenant context: business profile/policies, current business-state data relevant to the inquiry, and the customer's

prior conversation history (if any).

- FR-12: The agent checks live business-state data before confirming anything to a customer — a quantity, an availability, or a rate — and offers accurate, specific information (e.g., partial stock, current capacity, current rate) rather than a flat yes/no when relevant.
- FR-13: The agent drafts a response and scores its own confidence/risk level.
- FR-14: High-confidence, low-risk responses are sent automatically to the customer on the same channel they messaged from.
- FR-15: Low-confidence or high-risk responses (disputes, unusually large or high-value requests, ambiguous asks, anything outside defined policy) are routed to the owner via WhatsApp for approval or edit before sending, or the owner can respond directly and the system learns from it.

5.6 Memory & Learning

- FR-16: The system stores per-tenant conversation history, customer preferences, and business-specific facts, retrievable in future conversations.
- FR-17: Outdated or superseded information (old stock counts, expired availability status, resolved issues) is deprioritized/pruned over time rather than persisting indefinitely.
- FR-18: When a human edits or overrides an AI draft, that correction is captured and used to improve future responses for that tenant (tone matching, policy accuracy).

5.7 Dashboard

- FR-19: Owner can view a log of all conversations, including auto-resolved and escalated ones.
- FR-20: Owner can view current business-state data and edit it directly if desired, displayed appropriately for their business type (a stock table for a retailer, an availability/rate list for a service business).
- FR-21: Owner can view basic analytics: response time, auto-resolution rate, inquiry volume by channel, conversion-related metrics where determinable.
- FR-22: Owner can manage business profile/policy settings and connected channels.

---

6. Non-Functional Requirements

- NFR-1 (Isolation): No tenant's data must ever be queryable or visible from another tenant's context, enforced at the data-access layer, not just the UI.

- NFR-2 (Latency): Auto-resolved customer replies should be sent within a few seconds of the inbound message under normal load.
- NFR-3 (Reliability/Error handling): If an external dependency fails (business-state source, Qwen Cloud API, a channel API), the system degrades gracefully — e.g., escalates to the human rather than sending an inaccurate or broken reply.
- NFR-4 (Scalability): Architecture supports adding new tenants across any industry without code changes, and new channel adapters without touching core agent logic (modular channel layer). The business-state model must not require a schema change or new code path to onboard a new industry vertical.
- NFR-5 (Security): Channel credentials (WhatsApp tokens, email credentials) are stored encrypted per tenant; owner authentication uses standard secure practices (hashed passwords, session/token handling).
- NFR-6 (Observability): Every automated decision (auto-reply vs. escalate) is logged with enough context to audit why the system chose that path.
- NFR-7 (Deployment): Backend runs on Alibaba Cloud compute (Function Compute or ECS — final choice made at build time based on setup speed) with ApsaraDB for PostgreSQL as the database, with proof of deployment included in the submission.

---

7. Constraints

- Hackathon timeframe — feature set above is scoped to be realistically buildable, not a full commercial product.
- WhatsApp Business API and Meta Graph API both require business verification steps that can take time to approve — plan channel setup early, have a fallback (e.g., a WhatsApp sandbox/test number) if verification isn't complete by demo time.
- LinkedIn messaging automation is not realistically accessible within this timeframe and is excluded from scope.
- Team size/time limits how much of the "stretch" channel (Instagram/Facebook) can be fully wired vs. documented as an adapter pattern.

---

8. Assumptions

- Businesses onboarding already have *some* existing record of their business-state data (even if informal — a notebook, a spreadsheet, memory, or just what's in the owner's head) that they can transfer during the one-time onboarding step; the system does not need to infer this data from zero.

- Business owners have a smartphone with WhatsApp and can respond to short check-in messages, since this is the primary ongoing maintenance mechanism.
- For the hackathon demo, one or two seeded example tenants — deliberately from different industries — are sufficient to demonstrate both multi-tenancy and cross-industry generality; a large-scale multi-tenant load test is out of scope.
- Qwen Cloud and Alibaba Cloud services used are within free/trial tier limits sufficient for a demo-scale deployment.

---

9. Acceptance Criteria

The hackathon build is considered complete when:

☐ A new business can sign up, complete onboarding (profile + initial business-state data + at least one channel), and reach their dashboard — with a second, separate test tenant **from a different industry** demonstrating both data isolation and that the platform genuinely generalizes beyond stock-based businesses.
☐ A test customer message sent via WhatsApp (and via Email) is received, processed, and answered correctly with respect to live business-state data, without manual intervention, for a straightforward inquiry — tested against at least one product-based tenant and one service-based tenant.
☐ A test customer message that should be ambiguous/high-risk is correctly routed to the owner for review instead of being auto-answered.
☐ The owner receives at least one proactive WhatsApp check-in message about a business-state item and can update it via a plain-text reply, with the dashboard reflecting the change.
☐ At least one human correction to an AI draft is shown to measurably affect a later response (tone, detail, or policy) for that tenant.
☐ The dashboard displays conversation history, current business-state data, and basic analytics for the logged-in tenant only.
☐ The backend is verifiably running on Alibaba Cloud, with a code file in the repo demonstrating direct use of an Alibaba Cloud service/API.
☐ The repository is public, includes a visible OSS license, an architecture diagram, and documentation sufficient for a judge to understand and test the project without the original team present.

---

10. Resolved Decisions

- **Auth & database provider (reversed from earlier Supabase discussion):** Supabase runs on its own infrastructure, not Alibaba Cloud — using it would leave nothing

genuine to point to for the required Alibaba Cloud deployment proof. Decision: use **ApsaraDB for PostgreSQL** (Alibaba's managed Postgres) for the database, and a self-built JWT auth flow (bcrypt-hashed passwords, standard access/refresh tokens) rather than an external managed auth provider. This keeps auth, data, and compute verifiably on Alibaba Cloud, and avoids any "Clerk vs. Clerkey" naming confusion in code/docs since no external Clerk library is used.

- **Business-state data model:** generalized to a single flexible "business state item" shape (name, current value, last-confirmed timestamp — see FR-5b) rather than a fixed inventory-only schema, so retail, legal, agency, and consulting tenants all use the same underlying tables. Industry-specific onboarding prompts and dashboard labels sit on top of this shared model rather than requiring separate schemas per vertical.
- **Onboarding data source:** CSV upload / manual form entry during onboarding is the required baseline (no external dependency risk for the demo). A real connector for product-based tenants (e.g., Shopify) is a stretch goal only, attempted after the core flow is solid.
- **Proactive check-in trigger rule:** fires when EITHER (a) a business-state item receives 2+ customer inquiries within a rolling 24-hour window, OR (b) 48 hours have passed since that item was last confirmed by the owner — whichever happens first. Applies the same way whether the item is a stock count or a service availability flag.
- **Track submission:** Track 4 (Autopilot Agent) as the primary submission, with the memory/learning system positioned as a Track 1–adjacent strength in the written submission rather than a formal dual-track entry.